FPPVS Algorithms

Algorithm Description, Implementation Changes, and
Synthetic Comparative Evaluation

Submitted by
23CSB0B03
23CSB0B17
23CSB0B23
23CSB0B56

Study Setup and Scope

The scale comparison uses Poisson-arrival synthetic request streams (mean 3,997 requests per 18-hour stream; the three paired seeds produced 3,996, 3,976, 4,088 requests), an 80/20 training/test split, fully connected synthetic fog networks with 10, 30, and 100 nodes, three paired seeds (7–9), and 100 policy-training episodes. Reported acceptance, latency, resource and compute values are held-out means across seeds; each graph's caption states its sample and error-bar definition. The 2022 method is a local adaptation, not a reproduction using the paper's original request trace or full system model.

1. Original FPPVS Algorithm

The published Fairness-aware and Parallelized SFC Placement with VNF Sharing framework combines four mechanisms:

1. **SFC parallelization.** It transforms an ordered service function chain into a directed acyclic graph with a main chain and auxiliary branches. Monitor VNFs can be placed on branches; shaper VNFs remain ordered on the main chain. Dependency constraints preserve the service's packet-modification semantics. Each resulting sub-chain can then be routed and placed independently.
2. **VNF sharing with demand prediction.** The system first considers reuse of a compatible live instance, preferring candidates with sufficient remaining resources and favorable communication cost. If reuse is infeasible, it instantiates a VNF on a feasible node or rejects the request. A bounded SQN queue and LSTM summarize recent VNF demand and host/link placement history; its predicted distribution helps prioritize candidate hosts.
3. **Fair resource allocation.** A concave utility over flow throughput discourages a small number of requests from monopolizing shared resources. The fairness term is combined with cost, end-to-end latency, and service acceptance in the paper's weighted objective.
4. **Federated actor-critic learning.** Local agents observe placement state, prune nodes that cannot satisfy resource or bandwidth constraints, and select a feasible reuse or deployment action. Local model updates are periodically aggregated by a server using federated averaging. The paper also uses reward-adaptive exploration.

The published evaluation measures service acceptance, end-to-end latency, throughput, resource utilization, SFC assembly length, runtime, and Jain's fairness index. Its objective weights balance cost, delay, acceptance, and throughput utility. The paper's full design includes parallel-chain construction,

fairness logic, federated training, and SQN-LSTM prediction; the local comparison baseline below is only paper-style and must not be read as a complete reproduction of those components.

2022 Paper Adaptation (Brief Overview)

The separate adaptation models online SFC placement with sequential VNF placement, reuse of an existing same-type VNF, a TD = 20 SQN-LSTM queue to forecast demand, an actor-critic placement policy, and rollback when a request cannot be completed. Its reward follows an LTCR-based formulation with a failure-penalty approximation. Unlike updated FPPVS, it retains forecasting as part of its placement method.

2. Updates Made to FPPVS

The updated ours variant changes capacity handling and policy inputs while preserving a common evaluation objective:

1. **Reservation at VNF creation.** The local original-style implementation reserves 3× demand for a new shareable VNF. For d = 4, it reserves 12 units. Updated FPPVS initially reserves only the current request's demand, so d = 4 allocates 4 units. If another live request needs additional capacity for the same VNF type at that data center, the updated instance grows by the arriving request's demand, provided physical capacity allows it. The 3× multiplier is a local codebase setting, not a multiplier specified by the original paper.
2. **Expiry and release.** At request TTL expiry, updated FPPVS releases that request's instance allocation and shrinks the instance to the demand still in use. An idle on-demand instance holds zero compute capacity.
3. **Physical capacity enforcement.** Treat each data center as having a fixed compute budget (for example, 100 CPU units). Before accepting a placement, updated FPPVS accounts for pooled instance capacity, compute assigned to active requests, pending instances being started, and dedicated-instance reservations. If the combined total would exceed the physical limit, it rejects the placement. The usual original-style comparison does not enforce the same limit on pool reservations: it can record, for example, 110 units of pool capacity at a 100-unit data center and report a placement as feasible despite that overcommitment.
4. **Reuse and expansion.** Both variants reuse a matching shareable VNF type at the same data center when an existing instance has enough remaining capacity. Updated FPPVS can also expand an existing same-type instance when its spare capacity is insufficient. The original-style variant follows its fixed reservation rule for new instances.
5. **Idle-instance eviction.** When new capacity is needed, updated FPPVS can reclaim idle shareable instances. It evicts the least-recently-used idle instance first until the allocation can fit.
6. **VNF share classes.** The configured classes are IDS, DPI, load balancer, and NAT as shareable; firewall is dedicated. This distinction already existed in the repository's original model, so it is not a new difference between the two FPPVS variants.
7. **Dynamic traffic used in the experiments.** Request arrivals follow a homogeneous Poisson process over an 18-hour window (64,800 s) with rate *lambda* = 3,997 / 64,800 = 0.0617 requests per second (about 1 per 16.2 s). Inter-arrival gaps are therefore exponentially distributed with a mean of about 16.2 seconds, and the number of requests varies by seed (mean 3,997 expected; 3,996, 3,976, 4,088 for seeds 7–9). Arrivals are bursty rather than evenly spaced, and arrival times are rounded to whole seconds. Each request has a synthetic 1,000-second TTL, a randomly selected chain template containing 1–7 VNFs, compute demand of 1–8 units per VNF, traffic rate of 30–60 units, and a maximum-latency limit of 50–100 units. Endpoints and VNF templates are randomized.

Network links within a 10-node pod have 20–40 latency units; links between pods have 80–150, with bandwidth capacities of 1,200–1,800 units. The Poisson arrival process, the 1,000-second TTL and the generated trace are experimental assumptions, not the original trace.

Reporting and objective. Reports separate reserved instance capacity from compute in use by active requests; the cost term includes both normalized measures. Updated FPPVS uses the five-seed sweep's selected weights (cost 0.10, latency 0.25, acceptance 0.45, utility 0.20), evaluated under the same paper-weight objective as other candidates. This small sweep does not establish a global optimum.

3. Algorithm Comparison

3.1 Metrics and Interpretation

Service acceptance: accepted held-out requests divided by all held-out requests; higher is better. **Reserved instance capacity:** mean pooled VNF capacity held in the reported request snapshots; lower indicates less capacity tied up in the pool. **Request compute:** mean compute consumed by accepted requests; it is reported separately from reservation. **Accepted-request latency:** mean end-to-end latency for accepted SFCs only; lower is better.

The scale comparison uses the same synthetic workload and paired seeds, but the methods do not all share the same reservation or reward semantics. Therefore, acceptance and resource measures are more directly interpretable than raw accumulated reward across the 2022 adaptation and FPPVS variants. Greedy and random allocation are included for acceptance context; their resource and latency values were not recorded in this run.

3.2 Results Tables

Table 1. Held-out service acceptance (mean ± standard deviation across three paired seeds; higher is better).

Fog nodes	Original FPPVS	Updated FPPVS	2022 adaptation	Random allocation
10	45.03 ± 2.17%	26.36 ± 1.11%	44.57 ± 2.35%	21.82 ± 3.49%
30	1.20 ± 0.51%	3.07 ± 0.78%	1.52 ± 1.44%	1.87 ± 0.66%
100	0.54 ± 0.08%	0.66 ± 0.19%	0.54 ± 0.08%	0.13 ± 0.13%

Table 2. Mean instance capacity reserved (capacity units; lower means less pooled capacity reserved).

Fog nodes	Original FPPVS	Updated FPPVS	2022 adaptation
10	1195.8	143.7	1207.9
30	92.3	22.3	116.4
100	94.4	14.2	94.4

Table 3. Request compute in use per accepted request (compute units; distinct from instance capacity reserved).

Fog nodes	Original FPPVS	Updated FPPVS	2022 adaptation
10	12.24	8.68	12.25
30	6.48	7.98	9.09
100	10.27	11.84	10.27

Table 4. Mean latency of accepted requests (synthetic latency units; lower is better).

Fog nodes	Original FPPVS	Updated FPPVS	2022 adaptation
10	55.2	56.7	55.0
30	51.8	54.3	57.8
100	52.1	50.0	52.1

3.3 Why Acceptance Is Low

The low acceptance in the larger-network runs is mainly a consequence of the generated offered load meeting hard placement constraints, with policy quality also limiting how many feasible placements are found. The stream places a mean of 3,997 requests over 18 hours with Poisson arrivals (about 16.2 seconds apart on average); with a 1,000-second TTL, roughly 62 requests are active at once on average (the concurrent count itself is Poisson-distributed, so it fluctuates around 62 with a standard deviation near 8 and is burstier than under even spacing). A request averages about four VNFs, each demanding 1–8 compute units (mean 4.5), or roughly 18 compute units per request. That implies around 1,100 compute units of concurrent offered demand. By comparison, the average total data-center capacity is about 240, 720, and 2,400 units for 10, 30, and 100 nodes. Thus 10- and 30-node cases are heavily loaded relative to aggregate capacity.

At 100 nodes, total compute capacity alone does not explain the low rate. Each request must also satisfy its latency limit (50–100 units) and link bandwidth along the complete chain. The generated topology assigns 20–40 latency units to links within a 10-node pod and 80–150 across pods; inter-pod links can exceed a request's entire latency budget before other hops are counted. Placement decisions and only 100 training episodes further limit the learned policy: at 100 nodes, greedy accepts about 4.15% while updated FPPVS accepts 0.66%, suggesting feasible placements are not always found by the learned policy. The comparison's original-style baseline also permits pool overbooking; when its local 3× pool is physically constrained in a separate 10-node diagnostic (run earlier with evenly spaced arrivals and not repeated for this Poisson study), its acceptance falls from 45.5% to about 1.0%, while updated FPPVS is 26.4% in the main Poisson run.

The scale experiment does not record rejection counts by cause, so it cannot assign an exact share of failures to compute, latency, bandwidth, or policy decisions. The explanation above identifies the pressure points from the workload ranges, feasibility checks, and comparison runs; it is not a per-cause failure decomposition.

3.4 Where the Updated Algorithm Performs Better

- **Capacity control:** updated FPPVS reserves less pooled capacity than original FPPVS at every tested size: approximately 88.0% less at 10 nodes, 75.8% less at 30 nodes, and 84.9% less at 100 nodes. This is the most consistent advantage in the current measurements.
- **Admission:** it performs worse at 10 nodes (26.36% versus 45.03% original), but higher at 30 nodes (3.07% versus 1.20%) and 100 nodes (0.66% versus 0.54%). Those large-network absolute differences are small and have substantial seed variation.
- **Latency:** updated latency is slightly worse at 10 and 30 nodes and lower at 100 nodes. It is therefore not a consistent latency winner.

The 2022 adaptation is strongest or near strongest at 10 nodes, while updated FPPVS has higher acceptance at 30/100 nodes in this run. Random allocation is included as broader admission context in Figure 1.

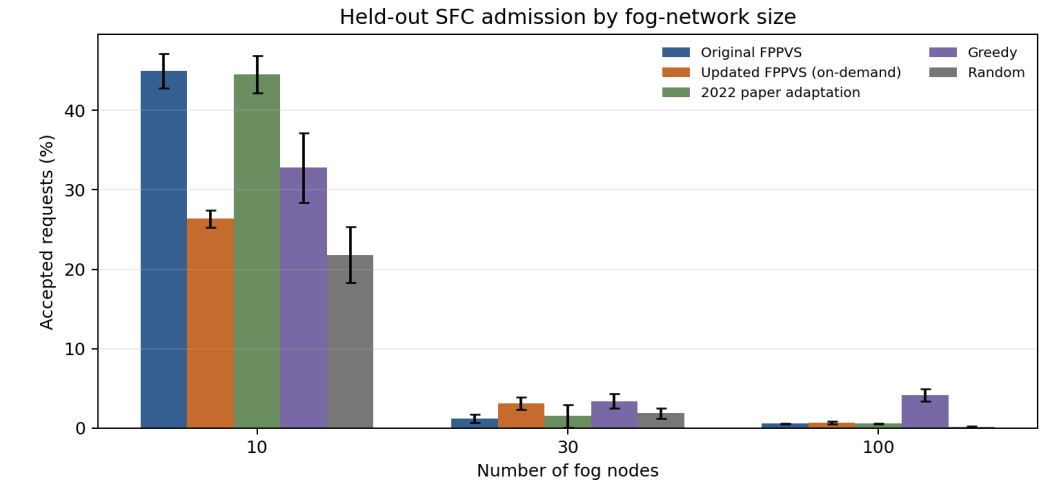
3.5 Important Fairness Limitation

The principal original-style baseline allows pooled reservation to exceed physical capacity, while updated FPPVS enforces the aggregate pool limit. Its higher admission at 10 nodes is therefore not a physically equivalent comparison. In a separate 1,000-episode diagnostic (evenly spaced arrivals; not repeated under Poisson arrivals), enforcing physical capacity on the local original-style 3× pool reduced its mean 10-node acceptance to about 1.04%. The local 3× rule is an implementation choice, so this diagnostic should not be interpreted as a direct comparison to the paper's published FPPVS results.

3.6 2022 Adaptation Comparability

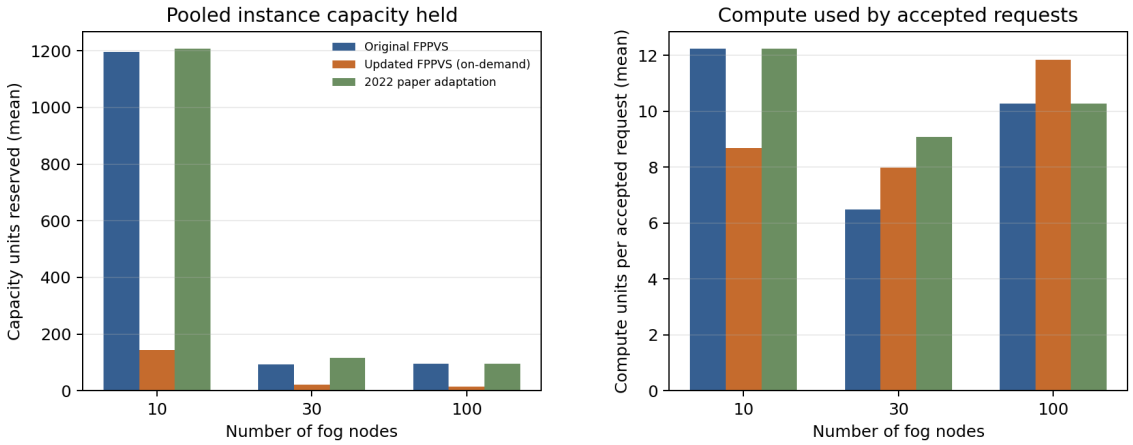
Its admission, latency, and resource use can be compared under this shared synthetic workload, but its internal LTCR reward should not be ranked against the FPPVS weighted reward. The adaptation's TD = 20 SQN and synchronous trainer are local modeling choices documented above.

4. Figures



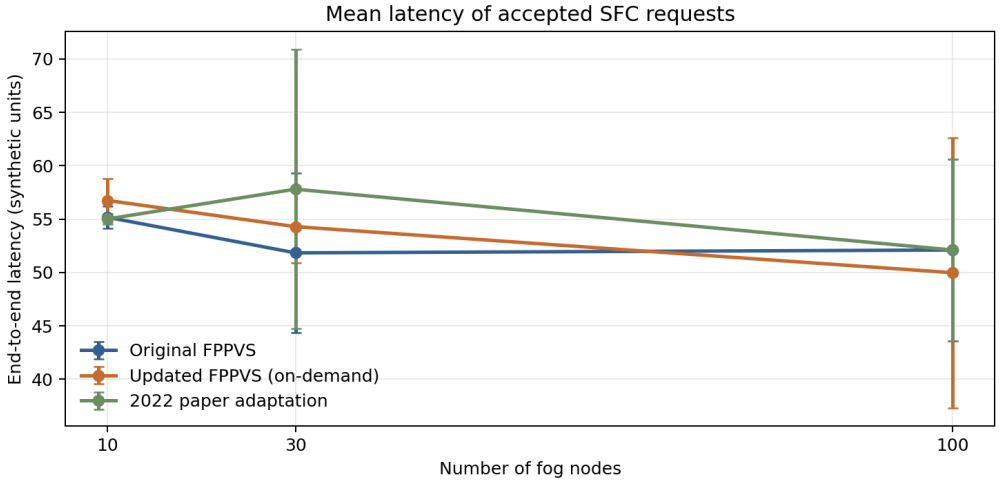
Source: synthetic Poisson-arrival experiment (mean 3,997 requests / 18 h); 80/20 split; 3 seeds per node count; 100 training episodes. Error bars show one standard deviation across seeds.

Figure 1. Held-out service acceptance. Updated FPPVS accepts fewer requests than original FPPVS and the 2022 adaptation at 10 nodes. It is modestly higher than original at 30 and 100 nodes. The high-node differences are small in absolute terms and should be read with the one-standard-deviation error bars and only three seeds in mind. Error bars are standard deviations across seeds.



Source: synthetic Poisson-arrival experiment (mean 3,997 requests / 18 h); 80/20 split; 3 seeds per node count; 100 training episodes. Pool capacity and request compute are separate measures.

Figure 2. Reserved capacity and request compute. The left panel shows substantially lower pooled capacity reservation by updated FPPVS at every network size. The right panel shows that compute used by accepted requests is not always lower: updated compute is lower than original at 10 nodes but higher at 30 and 100. The figure demonstrates why reservation and workload compute must be reported as separate quantities. Error bars are standard deviations across seeds.



Source: synthetic Poisson-arrival experiment (mean 3,997 requests / 18 h); 80/20 split; 3 seeds per node count; 100 training episodes. Error bars show one standard deviation across seeds.

Figure 3. Latency of accepted requests. Latency is close among the three learned methods. Updated FPPVS is about 1.6 units higher than original at 10 nodes, about 2.5 units higher at 30 nodes, and about 2.1 units lower at 100 nodes. This supports a size-dependent result, not a general latency improvement. Error bars are standard deviations across seeds.

5. Conclusion

The update replaces forecast-driven pooled sizing with reactive live-demand allocation, strict aggregate capacity enforcement, and explicit resource accounting. Under Poisson request arrivals the evidence still consistently shows lower reserved capacity. Acceptance and latency advantages depend on network size and comparator: updated FPPVS loses at 10 nodes, has a modest acceptance edge over original at 30/100

nodes, and offers no consistent latency gain. The 2022 algorithm's forecasting remains part of its method, while updated FPPVS no longer uses forecasts. Further claims of overall superiority require more paired seeds, larger training budgets, and validation on the original datasets and topologies.

References

[1] Q. Zhu, M. Li, and A. Rezaeipanah, “Fairness-aware and parallelized SFC placement with VNF sharing in MEC-enabled IoT networks,” *Computer Networks*, vol. 290, p. 112774, 2026.

[2] Zhang et al., “A dynamic planning model for deploying service functions chain in fog-cloud computing,” *Journal of King Saud University – Computer and Information Sciences*, vol. 34, pp. 7948–7960, 2022. DOI: 10.1016/j.jksuci.2022.07.012.